esprit
Se former autrement

UNIFIED MODELING LANGUAGE



Chapitre V:
Conception statique
-
Diagramme de classes de conception

Plan

- Conception détaillée des classes
- Les relations de dépendance
- Diagramme de classes : architecture en 3 couches

Objectif

- Passage du diagramme de classes d'analyse et des diagrammes d'interactions vers le diagramme de classes de conception

esprit
Se former autrement

UNIFIED MODELING LANGUAGE™

1- Conception détaillée des classes

Année Universitaire

2020-2021

Il s’agit de :

- Spécifier les visibilités des attributs et des méthodes
- Définir les types des attributs identifiés en analyse
  - Type de base
  - Structure
  - Enumération
- Spécifier les méthodes
- Montrer les dépendances

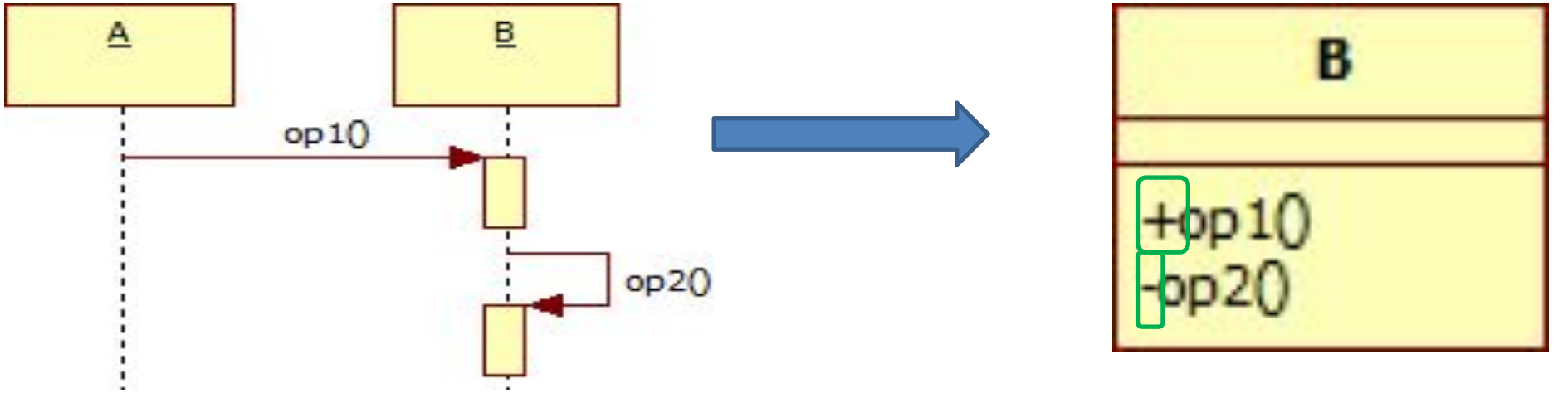
1- La visibilité des attributs et des méthodes

- Visibilités :
  - Public (+) : visible pour toutes les classes
  - Private (-) : n'est visible que pour la classe elle-même
  - Protected (#) : visible pour la classe et toutes les sous-classes
  - Package (~) : visible par les classes contenues dans le même package

- Visibilité par défaut :
  - Attribut : privé
  - Opération : public

1- La visibilité des attributs et des méthodes

- Exemple :
  - La classe B correspond à l'objet B du diagramme de séquence objet



2- La spécification des attributs

- Format de description d'un attribut :

[Vis] Nom [Mult] [":" TypeAtt] ["=" Val]

  - Visibilité : + (public), - (privé), # (protégé), (package)
  - Multiplicité : l'attribut représente un ensemble de valeurs: [" nbElt "] ou [" Min .. Max "]
  - Type de l'attribut : type primitif (Entier, Chaîne, ...) ou classe
  - Valeur initiale : valeur prise par l'attribut lors de l'instanciation de la classe

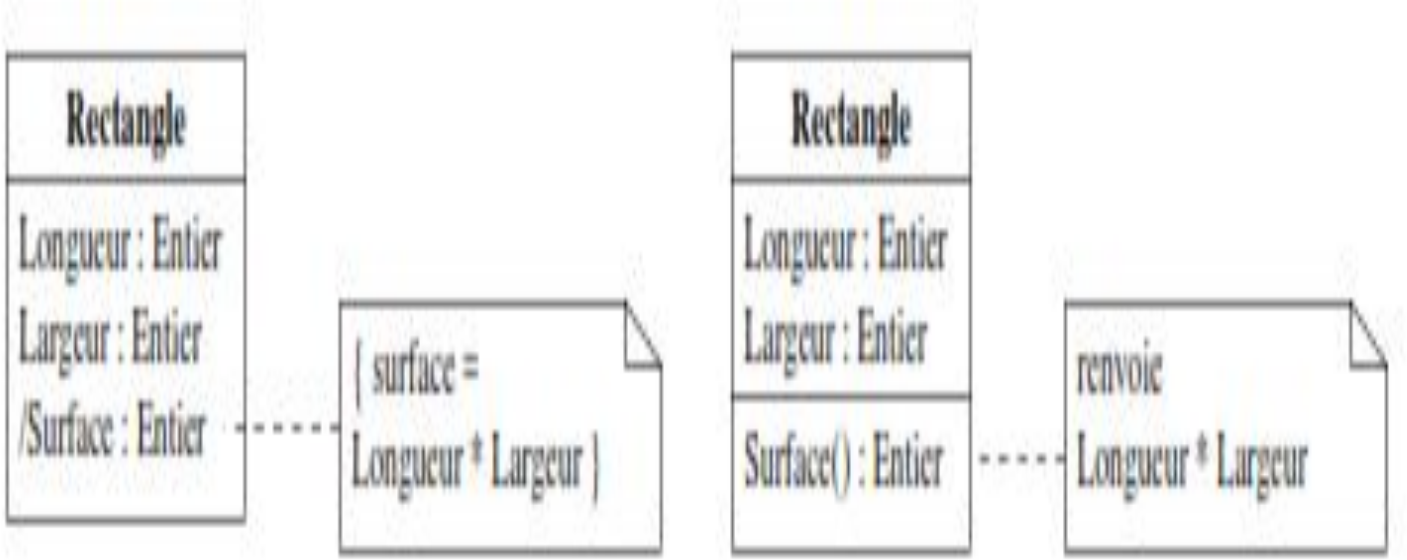
- Attributs dérivés précédés de "/«
- Attributs « Static » doivent être soulignés en UML

3- L’ attribut dérivé

- Les propriétés entièrement dépendantes d’autres propriétés peuvent être exprimées à l’aide d’attributs dérivés.
  - Les attributs dérivés sont calculables à partir d’autres attributs
- Un attribut dérivé peut être traduit par:
  - **un attribut et une opération** qui met à jour et retourne la valeur de l’attribut à chaque appel
  - **une opération** qui calcule la valeur lors de chaque appel (sans stockage de l’attribut)

    => selon la fréquence d’usage

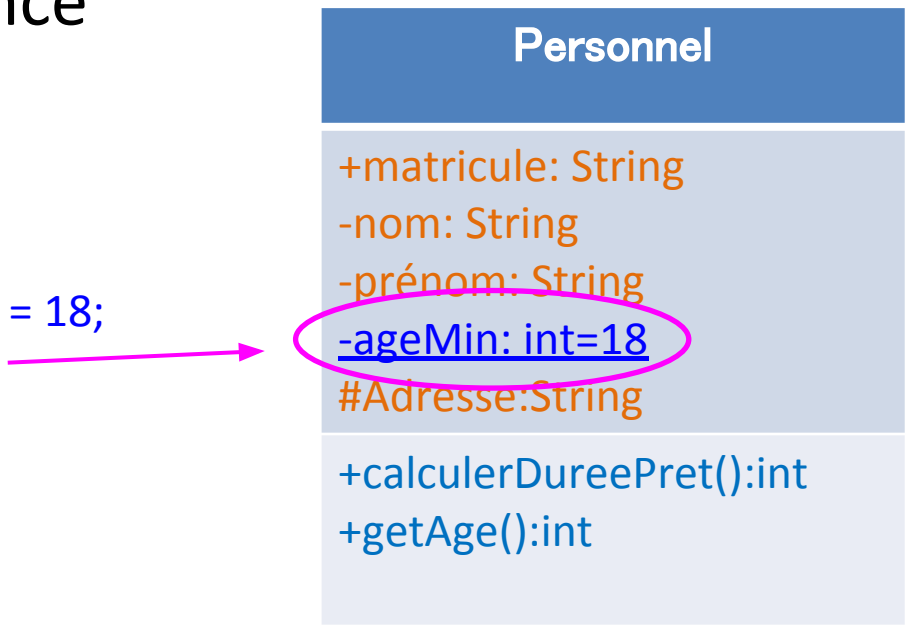
3- L’ attribut dérivé



4-L'attribut « Static »

- Un attribut ou une opération statique (attributs de classe) est un attribut ou une opération appartenant à une classe plutôt qu'aux instances de la classe => son existence est indépendante de l'instance

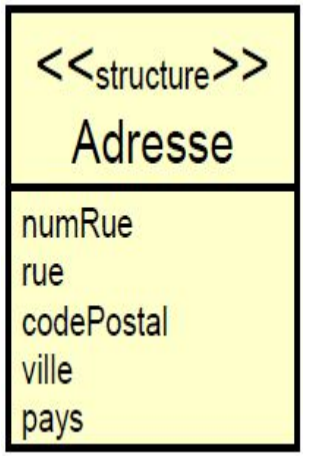
private static int ageMin = 18;



4- Les types des attributs : La « Structure »

- Ensemble d'attributs pouvant être regroupés

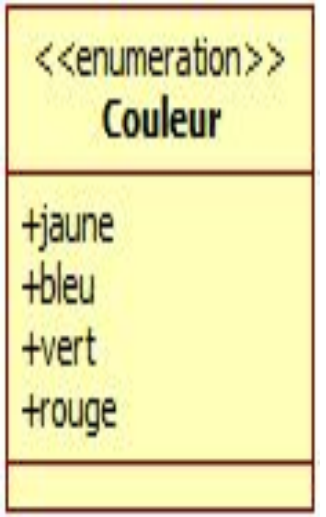


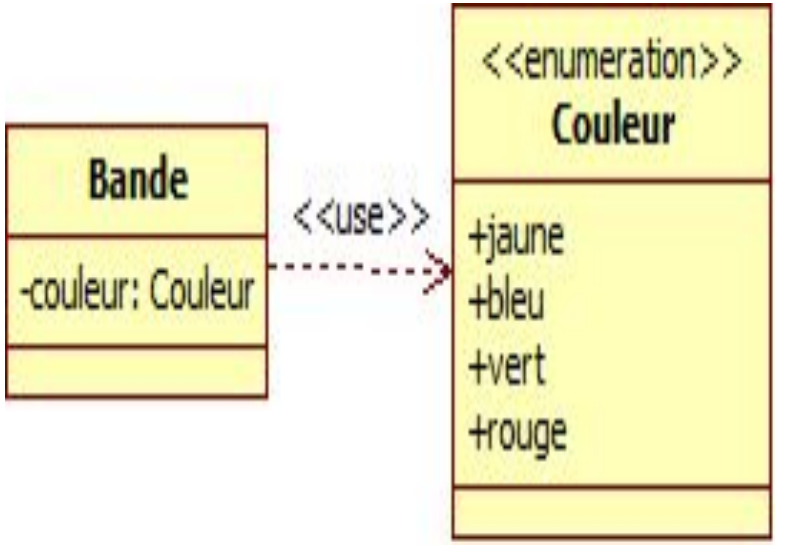


4- Les types des attributs : L' « Enumeration »

- Type énuméré : type possédant un nombre finis de valeurs possibles

- Exemples:





5- La spécification des opérations

- Format de description d'une opération :

  [Visibilité] Nom ["(" Arg ")"] [":" Type retourné]

  – Visibilité : + (public), - (privé), # (protégé)

  – Arg : liste des arguments selon le format suivant:

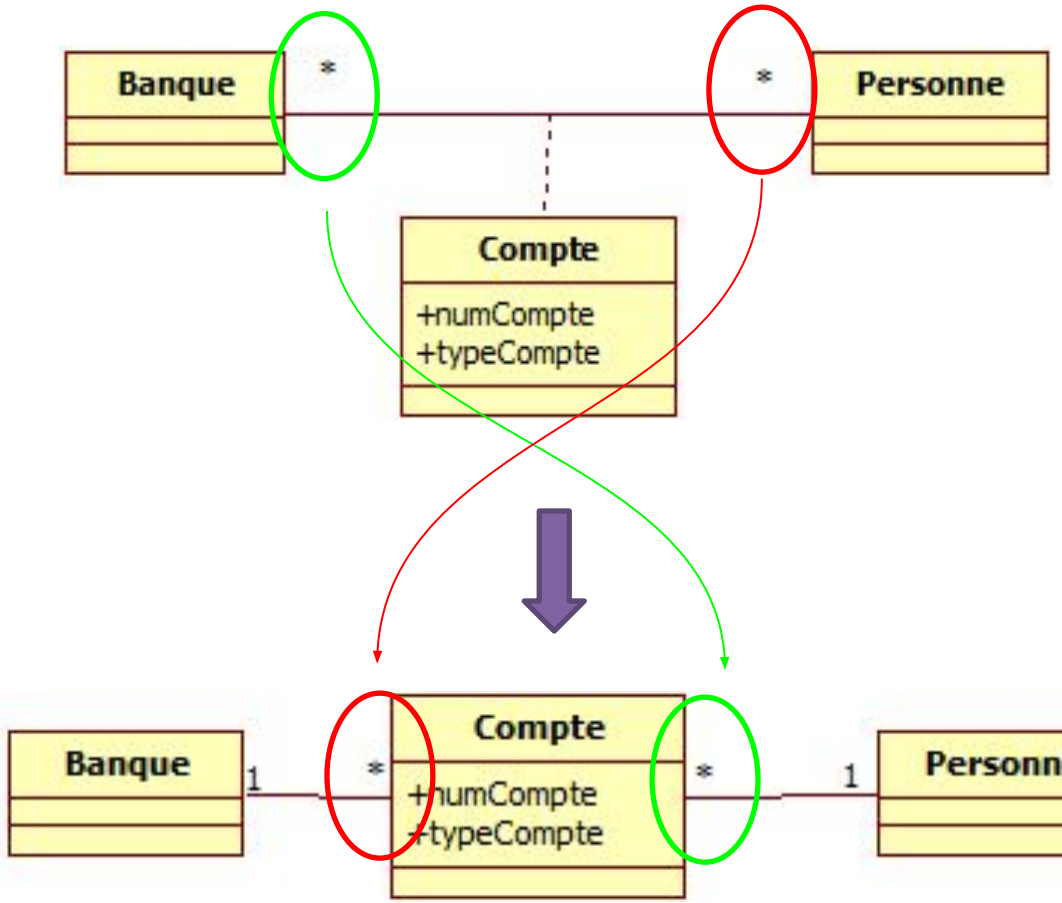
  [Dir] NomArgument : TypeArgument où Dir = in (par défaut), out, ou inout

  – Type : type de la valeur retournée (type primitif ou classe)

- Méthode « Static » / Méthode de classe (son existence est indépendante de l'instance) doit être souligné en UML

6- La transformation de la classe association

- La transformation la plus courante en conception:
  - La classe d'association devient une classe intermédiaire
  - L'association initiale est découpée en deux



7- La navigabilité d'une association

- C'est la capacité d'une instance de C1 (resp. C2) à accéder aux instances de C2 (resp. C1)
  - Elle permet de spécifier dans quel sens il est possible de traverser l'association à l'exécution.

- Par défaut une association est bidirectionnelle
  - Navigation dans les deux sens

- On restreint la navigabilité d'une association à un seul sens à l'aide d'une flèche.



7-La navigabilité d'une association/implémentation




```
public class Client {

public int Id_client;

private list<Reclamation>   reclamations
…
}
```

```
public class Reclamation {
Public int reference;
private Client client;
…
}
```

7- La navigabilité d'une association/implémentation




```
public class Client {

public int Id_client;

private list<Reclamation>   reclamations
…
}
```

```
public class Reclamation {
Public int reference;
~~private Client client;~~
…
}
```

7- La navigabilité d'une association/implémentation




```
public class Client {

public int Id_client;

private list<Reclamation>  reclamations
…
}
```

```
public class Reclamation {
Public int reference;
private Client client;
…
}
```

8- La classe « interface** »: Définition

- Description d'un ensemble d'opérations utilisées pour spécifier un service offert par une classe.

- Elle ne contient ni attribut, ni association, ni implémentation des opérations (les opérations sont abstraites)

- Une classe réalisant une interface doit implémenter les opérations de l'interface,

** Attention: Cette définition ne concerne pas les interfaces fonctionnelles dans JAVA 8

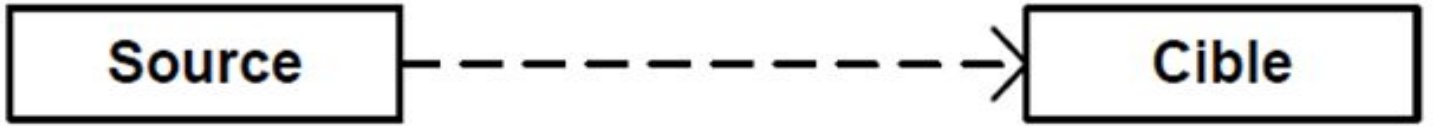
esprit
Se former autrement

UNIFIED MODELING LANGUAGE™

2- Les relations de dépendances

1- Définition : Relation de dépendance

- Relation unidirectionnelle indiquant qu'un changement dans la cible (fournisseur) provoque un changement dans la source (client).



- Types de relations de dépendances :
  - Abstraction: relation entre éléments qui représentent un même concept à différents niveaux d'abstraction;
  - Permission: l'élément source a le droit d'accéder à l'espace de nommage de l'élément cible;
  - Utilisation: l'élément source requiert la présence d'un élément cible.

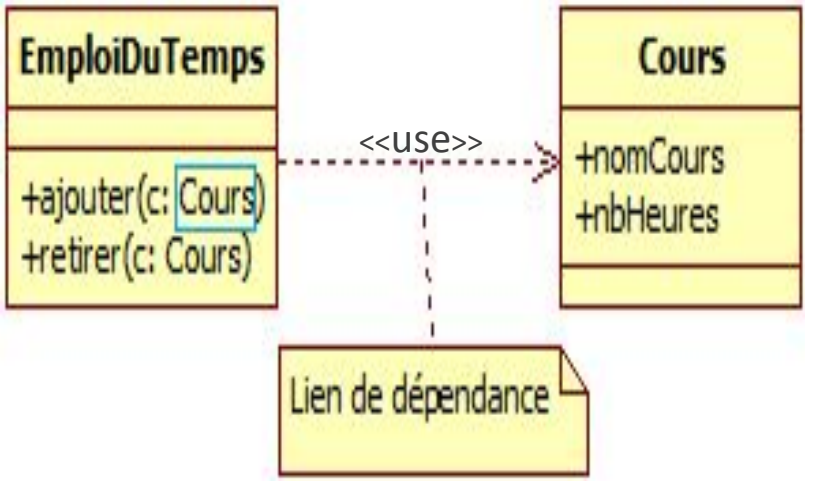
2- Les types de relations de dépendance

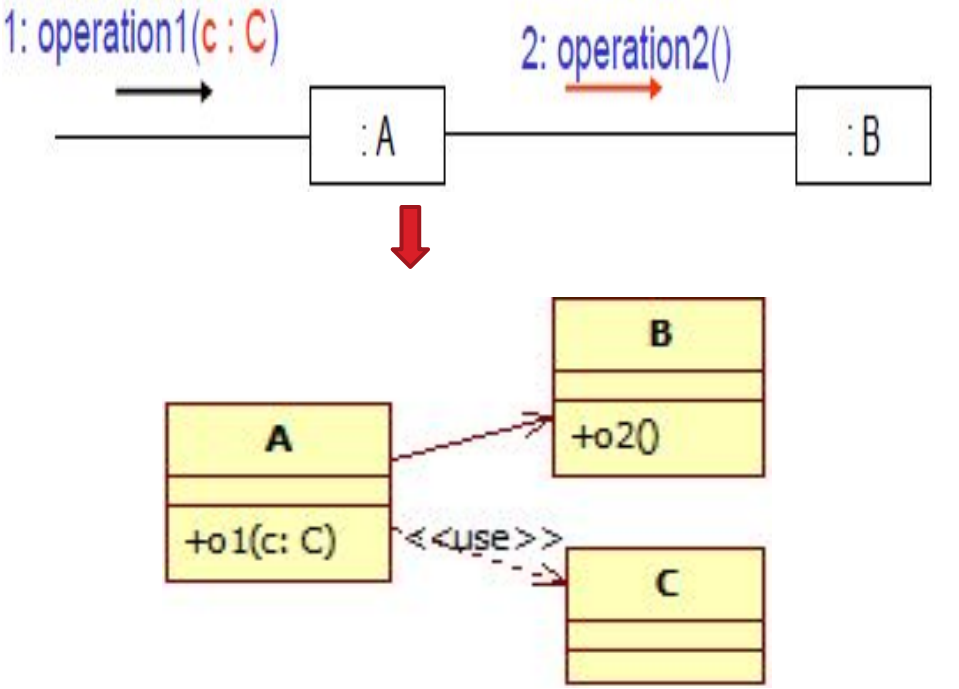
- **Pour l'utilisation :**
  - **« use »:** la source a besoin de la cible pour être implémentée
  - **« call »:** une opération source appelle une autre opération cible

- **Pour la permission :**
  - **« permit »:** le source est amie de la cible; elle permet à l'élément source d'ignorer la propriété de visibilité de l'élément cible
    - Exemple : la classe « Amie » en C++

- **Pour l'abstraction :**
  - **« realize »:** l'élément source implémente la spécification désignée par l'élément cible.

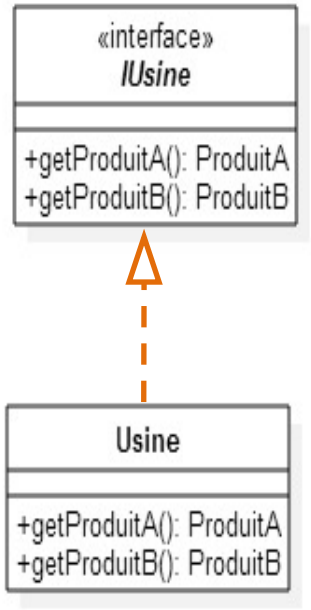
Exemples (1/2)





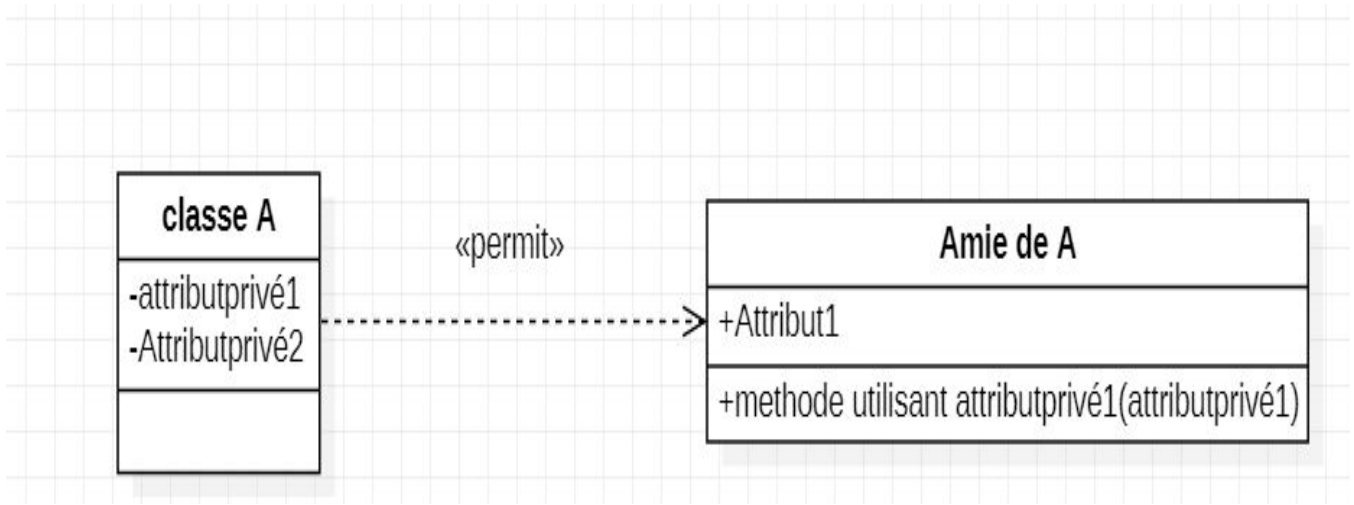
La méthode ajouter() de la classe « EmploiDuTemps » utilise un objet de la classe cours comme paramètre grâce à la relation « use »

Exemples (2/2)



La relation de dépendance « realize »:

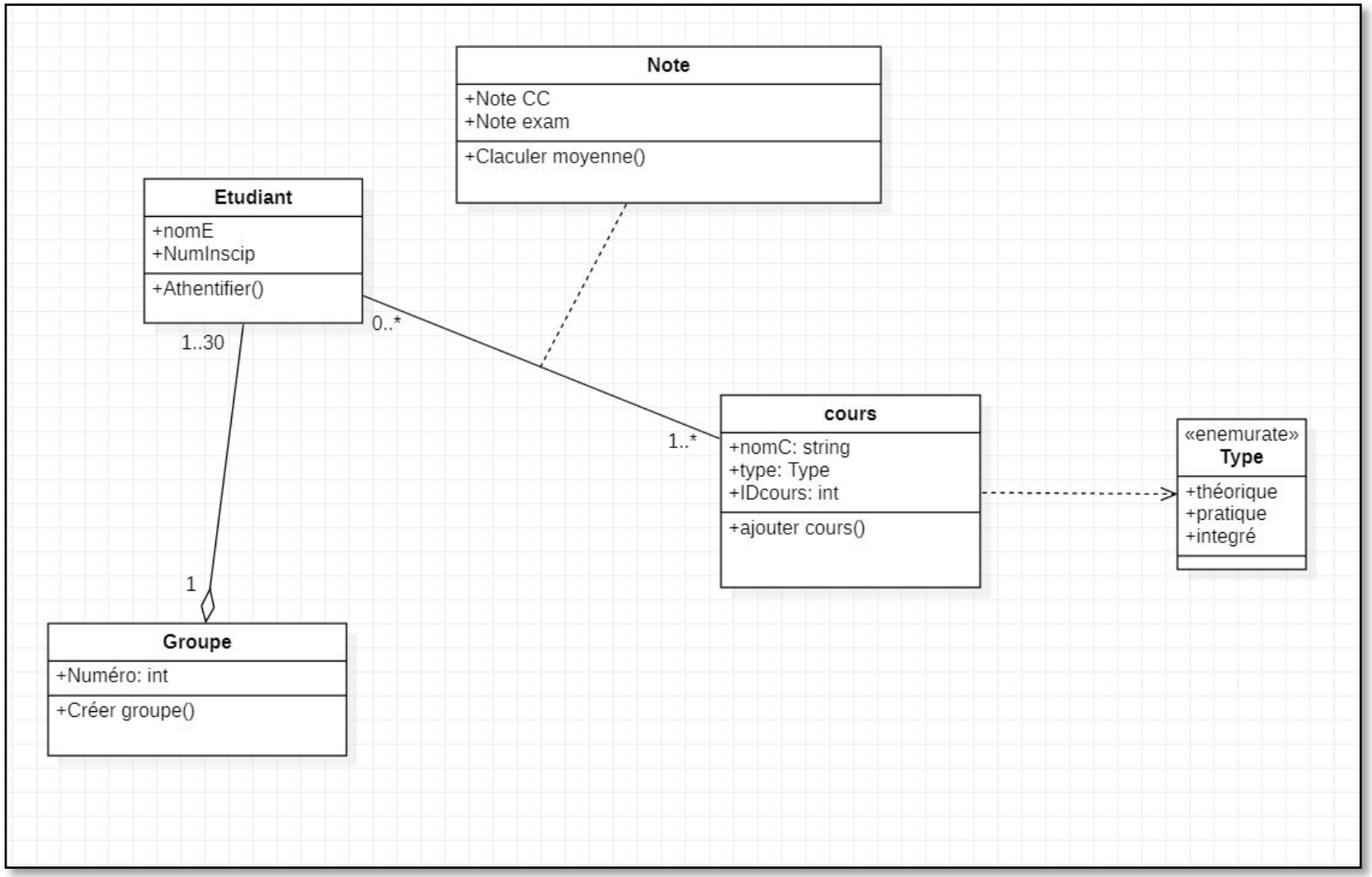
La classe Usine implémente l'interface IUsine



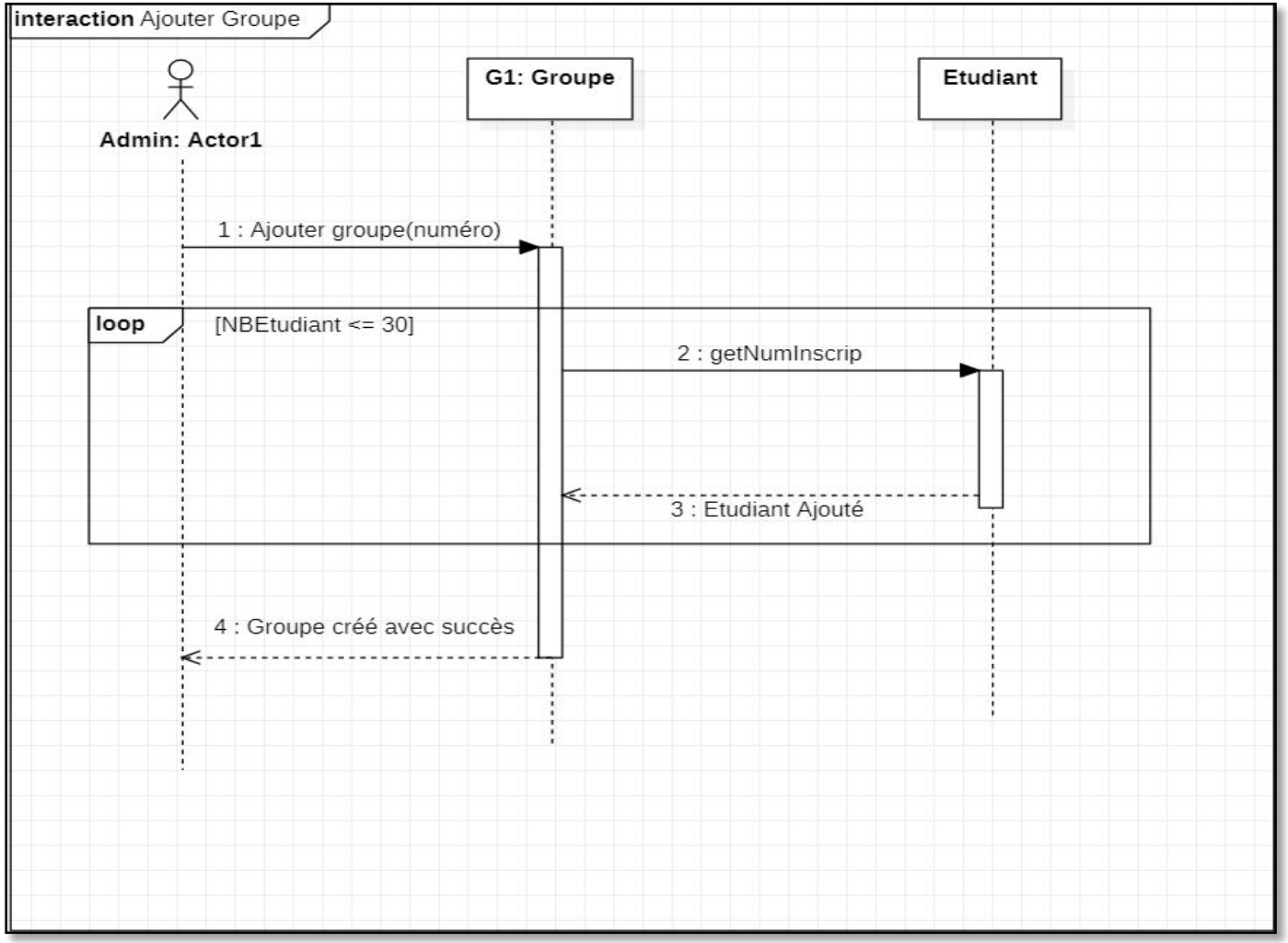
La relation de dépendance « permit »:

La classe « Amie de A » peut accéder à l'attribut « attributsprivé1 » malgré que cet attribut est private grâce à la relation permit

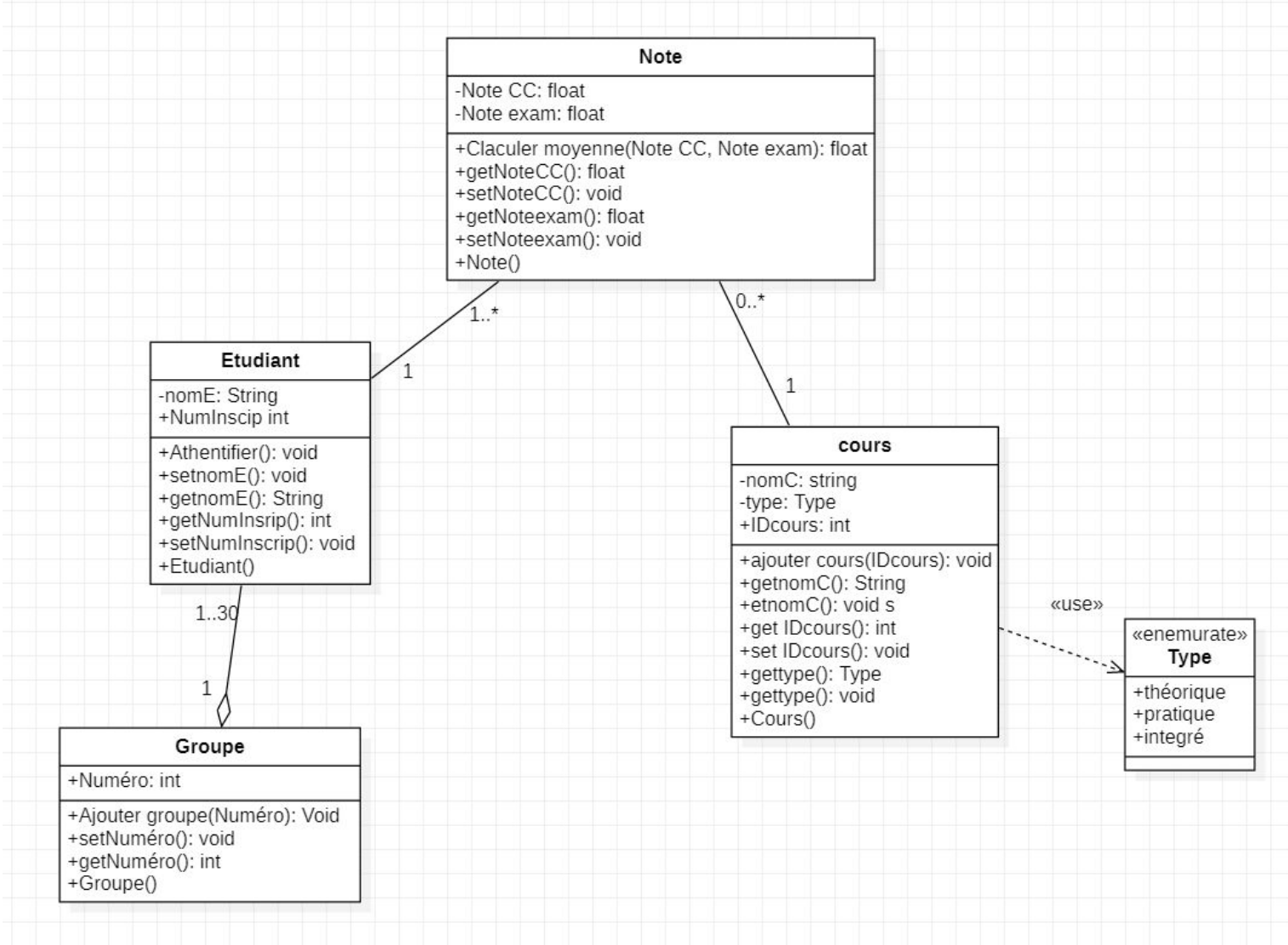
Etude de cas (1/4): Diagramme de classes d'analyse



Etude de cas (2/4): Extrait du diagramme de séquence objet " Ajouter Groupe"



Etude de cas (3/4): Diagramme de classes de conception (monocouche)



esprit
Se former autrement

UNIFIED MODELING LANGUAGE



3- Diagramme de classes : architecture en 3 couches

2- Classe graphique

- Les classes graphiques modélisent les interactions entre le système et ses acteurs.



2- Classe gestionnaire

- Un contrôleur est généralement classe de méthode qui est utilisée pour accepter les requêtes et renvoyer une réponse

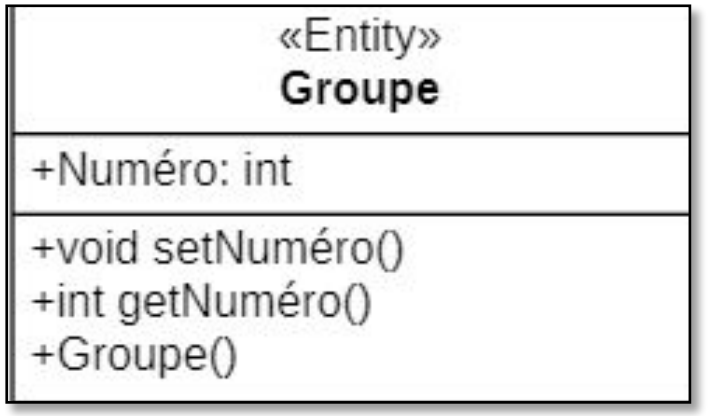
- Elle assure la coordination, l'enchaînement et le contrôle d'autres objets

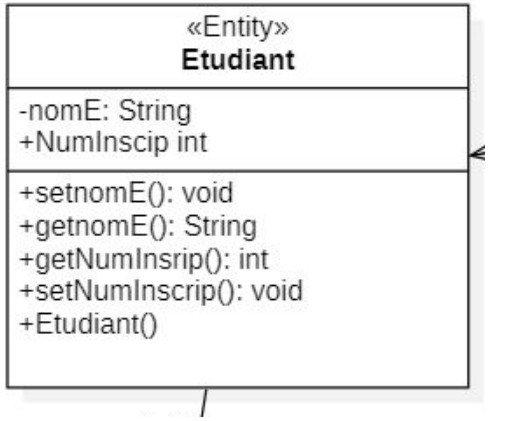
- Exemple de classes Gestionnaires: Les controllers (Symfony) , les services (java)…



2- Classe entité

- La classe entité modélise les informations persistantes du système.
- Exemples : classes qui contiennent des informations d'un étudiant, d'un employé, …

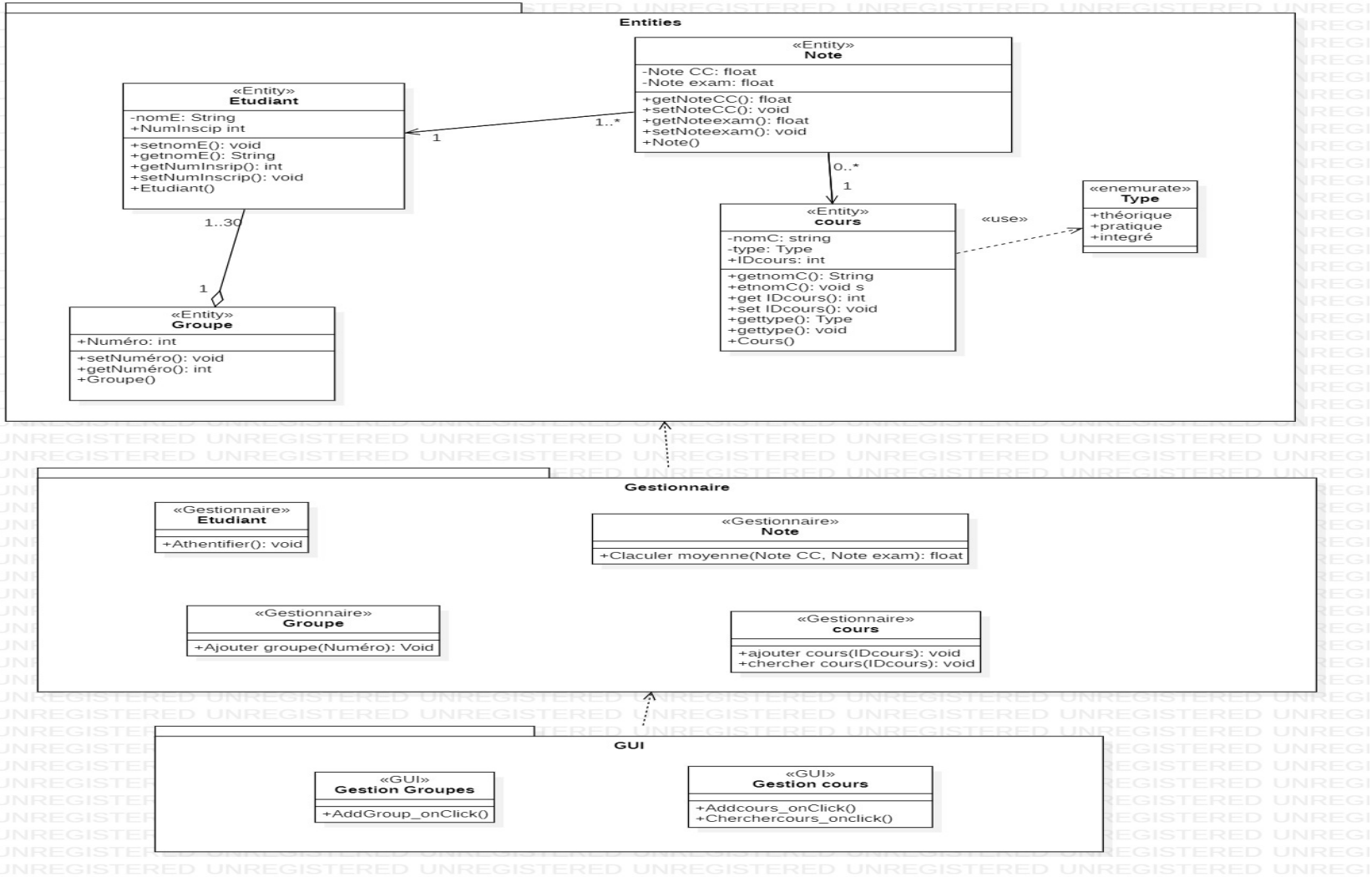




3- Associations entre classes

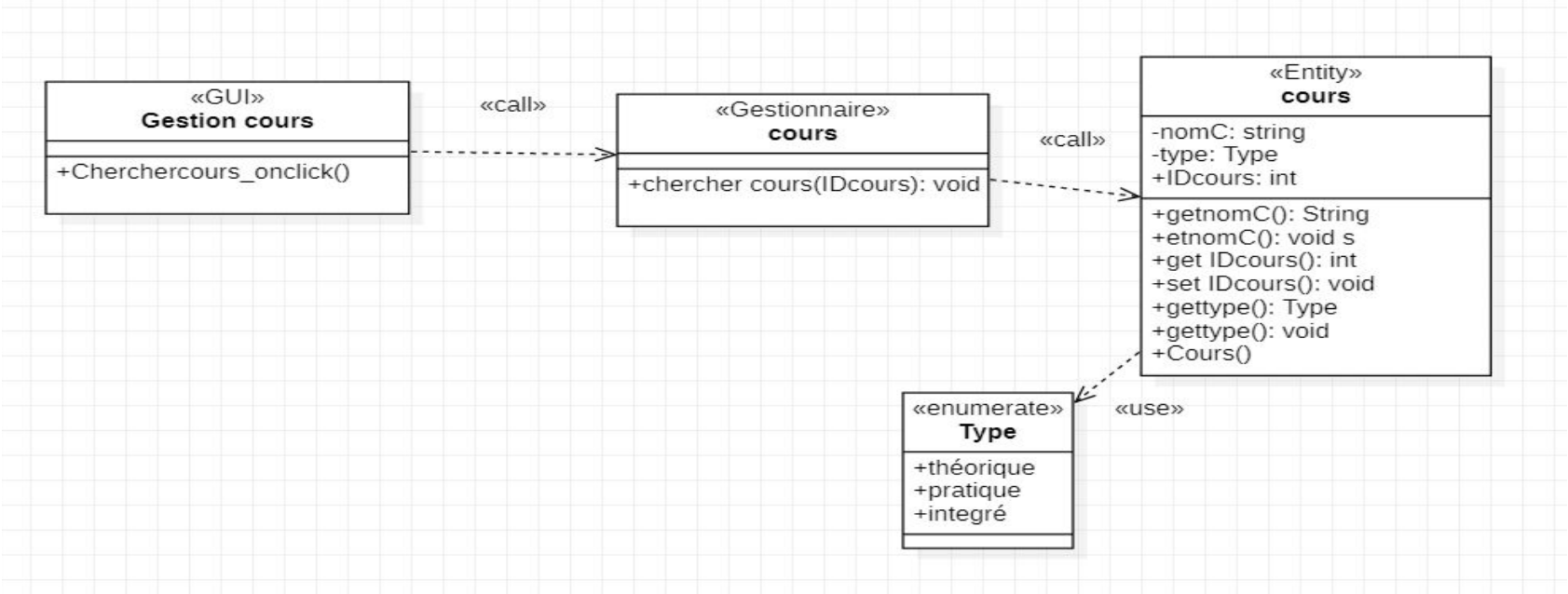
- Les associations entre les classes suivent des règles assez strictes :
  - Les classes graphiques ne peuvent être reliées qu'aux classes gestionnaires.
  - Les classes gestionnaires ont accès aux classes graphiques, aux classes entités et aux autres classes gestionnaires
  - Les classes entités ont accès aux autres classes entités et ne sont reliées qu'aux classes gestionnaires

Etude de cas (4/4): Extrait du diagramme de classes de conception (3 couches)



Extrait du diagramme de classes de conception

- Remarque: Puisque le diagramme de classes de conception en 3 couches est souvent très détaillé, on peut parfois se limiter à représenter ***un extrait du diagramme de classes de conception*** modélisant un seul cas d'utilisation.
- Exemple: Extrait du diagramme de classes de conception relatif au cas d'utilisation « Chercher cours »

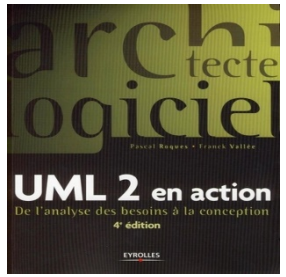


Références bibliographiques

- UML 2 par la pratique , Edition 7, Pascal Roques



- UML 2 en Action, Edition 4, Pascal Roques, Frank Vallée



- Annexe : Préparation de l'implémentation, Partie 2, Equipe UML, UP GL-BD

